



A Lagrangian heuristic for sprint planning in agile software development



Marco A. Boschetti^{a,*}, Matteo Golfarelli^b, Stefano Rizzi^b, Elisa Turrinchia^b

^a University of Bologna, Department of Mathematics, Via Sacchi 3, 47521 Cesena, Italy

^b University of Bologna, DISI, Mura Anteo Zamboni 7, 40126 Bologna, Italy

ARTICLE INFO

Available online 19 September 2013

Keywords:

Mixed integer linear programming
Software engineering
Agile methods
Optimization models
Matheuristics
Lagrangian heuristics

ABSTRACT

Agile methods for software development promote iterative design and implementation. Most of them divide a project into functionalities, called *user stories*; at each iteration, often called a *sprint*, a subset of user stories are developed. The sprint planning phase is critical to ensure the project success, but it is also a difficult problem because several factors impact on the optimality of a sprint plan, e.g., the estimated complexity, business value, and affinity of the user stories to be included in each sprint. In this paper we present an approach for sprint planning based on an integer linear programming model. Given the estimates made by the project team and a set of development constraints, the optimal solution of the model is a sprint plan that maximizes the business value perceived by users. Solving to optimality the model by a general-purpose MIP solver, such as IBM Ilog Cplex, takes time and for some instances even finding a feasible solution requires too large computing times for an operational use. For this reason we propose an effective Lagrangian heuristic based on a relaxation of the proposed model and some greedy and exchange algorithms. Computational results on both real and synthetic projects show the effectiveness of the proposed approach.

© 2013 Elsevier Ltd. All rights reserved.

1. Introduction

To overcome the problems of traditional software engineering approaches, one of the most promising directions seems to be agility, as suggested by some empirical studies [2,15]. Agile methods, such as *Scrum* and *eXtreme Programming* (XP), are based on the twelve principles stated in the Agile Manifesto [5] and have been adopted by an increasing number of companies aimed at making the software development process faster and nimbler. The Scrum approach is deeply discussed in [22], where its key ideas and its life-cycle are described.

A key practice shared by all agile methods is incremental and has iterative design and implementation: the software is described in terms of detailed user functionalities (*user stories*), and at each iteration (*sprint*) the team should deliver the set of user stories that maximize the utility for the users while fulfilling a set of constraints, such as limiting the duration of an iteration and respecting dependencies and correlation among user stories. According to the *team awareness* principle, sprint planning is based on sharing and averaging the estimates given by team members about story complexity, utility, non-delivery risk, and

dependencies. For example, advancing high-valued stories could lead to an early significant result for users and encourage the team awareness; similarly, critical user stories can be advanced to avoid late side-effects, but at the price of a higher probability of delays in the initial stages. The work in [12] focuses on user stories and gives practical hints for estimating their complexity and business value.

It is widely agreed that an effective sprint planning is one of the ingredients for a successful agile project [13]. The plan effectiveness depends in turn on the accuracy of estimates, and on the capability of properly taking all variables and constraints into account. While the first issue is mainly related to the team experience, the second one can be formulated as an optimization problem whose complexity increases with the project size. Clearly, a non-optimal solution to this problem leads to inefficiencies that easily turn into extra-costs and project delays.

Some commercial tools are available to support agile project management. AgileFant [1] offers a set of basic functionalities to monitor the progress of project iterations. *Mingle* [26] and *Scrum-Works* [14] provide a more complete set of parameters to deal with user story risk, complexity, and business value. However, all these solutions lack in providing an automated solution to the sprint optimization problem. Even some research prototypes have been developed in this context; for instance, [18] proposes an evolutionary and iterative approach that optimizes the requirement allocation to *increments* according to precedence constraints, priorities determined by stakeholders, and balance between

* Corresponding author. Tel.: +39 0547338863.

E-mail addresses: boschett@csr.unibo.it, marco.boschetti@unibo.it (M.A. Boschetti), golfare@csr.unibo.it (M. Golfarelli), stefano.rizzi@unibo.it (S. Rizzi), elisa.turrinchia2@unibo.it (E. Turrinchia).

required and available efforts. Recently, [25] and [27] proposed two mathematical models whose core is a *multiple knapsack*, with different objective functions and side constraints. However, both approaches maximize the *business value* of the deliverables taking into account story sizes and precedence relations. [27] also consider the user story completion probabilities, the uncertainty in velocity prediction, and *themes* (groups of functionally related stories) providing further business value. Both the models proposed by [25] and [27] are solved by branch and bound algorithms.

1.1. Our contributions

In a previous work [16,17] we formalized the sprint planning problem for agile projects and proposed an optimization model that, given the team estimates and a set of development constraints, produces an optimal sprint plan by maximizing the business value perceived by users. The proposed model was solved to optimality by a general-purpose MIP solver. Unfortunately, for difficult instances, even a powerful MIP solver such as IBM Ilog Cplex [19] is time consuming, and it turned out that in some cases even finding a feasible solution requires too large computing times for an operational use [17].

In this paper we address this efficiency issue following a *matheuristic* approach [7–9]. We propose an effective Lagrangian heuristic based on a relaxation of the model presented in [16], and some greedy and exchange algorithms. The proposed approach is tested on both real-world and synthetic projects and compared with the IBM Ilog Cplex solver. The Lagrangian heuristic is shown to provide small computing times and solutions of good quality. Therefore, it can be integrated into an optimization module that could be used interactively and coupled with existing softwares for agile project management.

1.2. Outline

The remainder of the paper is organized as follows. Section 2 summarizes the key agile practices with particular attention to Scrum and XP, two of the most commonly adopted methods, and gives a complete problem description. Section 3 gives some

definitions and notation, and proposes a mathematical model with some reduction procedures and valid inequalities. Section 4 presents two greedy heuristics and an exchange procedure, whereas Section 5 proposes a Lagrangian heuristic. Section 6 presents a set of tests on both synthetic and real projects to prove efficiency and effectiveness of our approach. Section 7 draws the conclusions and sketches some future work directions.

2. Problem description

Since one of the factors that determine the success of a project is customer satisfaction, agile methods aim at better complying with user requests by reducing the delivery time and making the development process more flexible. Indeed, accelerating the time-to-market leads to relieve the market pressure, while flexibility ensures fast reactions to both technology evolution and user requirement changes. To achieve these goals, agile methods propose several complementary practices, such as incremental and iterative development, user involvement, team awareness, and lean documentation. Several agile methods based on these practices have been arising from the practitioners' experience. As shown in [28], the Scrum method and some hybrid approaches (combining Scrum and XP practices) are preferred by companies. While Scrum focuses on managerial and processing aspects, XP defines specific techniques (such as pair programming) to make the coding phase more efficient.

Fig. 1 shows the typical life-cycle of a Scrum project. In the *user story definition* phase, the team and the users sketch the overall structure of the system and define a set of user stories. A user story is a relatively small piece of functionality valuable for users [12]; it represents a light specification that can be later detailed thanks to a continuous communication with the user, but at the same time it must be sufficiently described to enable its development complexity to be estimated.

During the *user story estimation* phase, a *utility* and a *complexity* are assigned to each user story based for instance on expert advice and similarity to past activities, or using consensus-based techniques such as planning poker. Utility is the business value of a user story as perceived by the user who defines that story.

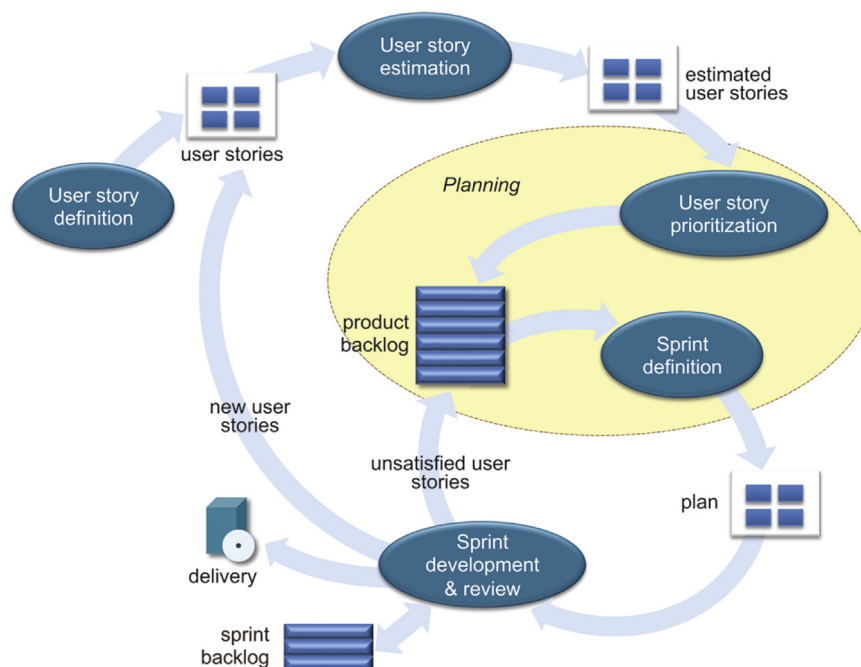


Fig. 1. Scrum life-cycle.

Some approaches only require to define an ordering for user stories (e.g., story 1 is more useful than story 2), but in general utility can be quantified through a positive numerical score. In this work we estimate utility with a score ranging between 10 and 100. The development complexity of a user story is measured in *story points*. Story points are non-dimensional and are preferred to time/space measures to avoid subjective and incomparable estimates. Team members assign story points to each story based on their experience and knowledge of the domain and project specificities. Several estimating methods were proposed [13,21,23]; some use numeric sizing (e.g., using the Fibonacci sequence 1, 2, 3, 5, 8, 13, etc.), others use non-numeric sizing (e.g., t-shirt sizes such as XS, S, M, L, XL, XXL). In this work we adopt numeric sizing, in the range [1, 10], that can be more easily accommodated within our numeric optimization model.

During the *user story prioritization* phase, the team assigns a priority to each user story, based on its utility and *risk*, and defines possible *dependencies* between stories. Risk is typically related to two different characteristics of user stories. A *critical* story is one that has a strong impact on the other stories, so that taking a wrong solution for it can dramatically affect the success of the project. An *uncertain* story is one for which it is somehow hard to estimate the complexity due to unexpected problems that could arise (e.g., user requirement changes and implementation problems). Both types of risk are normally estimated by positive numbers; in this work we adopt four classes of risk: 1 (no risk), 1.3 (low risk), 1.7 (medium risk), and 2 (high risk). Dependencies are development constraints indicating that a user story (post-condition) cannot start before one or more other user stories (pre-condition) are completed. Though agile methods discourage user story relationships to improve the project flexibility, some dependencies must necessarily be preserved [12]. In this work we distinguish conjunctive (AND-type) dependencies that imply that all precondition stories must be completed, and disjunctive (OR-type) dependencies that imply that at least one of the precondition stories must be completed.

The list of user stories resulting from prioritization composes the *product backlog* that is then partitioned into sprints during the *sprint definition* phase. Sprints should be short and regular enough to guarantee a prompt feedback from users (usually from one to four weeks). Allocation of user stories to sprints is based on the *development velocity*, on the complexity of stories, and on the possible correlations (*affinities*) among stories. The development velocity is the (estimated) number of story points the team can deliver per day; it is used to convert the expected sprint duration into the sprint *capacity*, meant as the maximum number of story points the team can deliver in a sprint. The affinity issue arises in the presence of correlated stories that have higher utility if they are included in the same sprint, because users better perceive the overall business value of the functionality delivered. For instance, a “zoom-out” story may have low utility on its own, but its utility may increase if delivered together with the complementary “zoom-in” story. The affinity range we adopt in this work is [0, 0.5], meaning that the utility of a story can be increased at most by 50%; this range has been empirically determined together with the agile design team involved in the case study we discussed in [16], because it allows for significantly but not disruptively impacting the utility of a story.

During each sprint (*sprint development* phase), the developers carry out a detailed analysis of the user stories involved, producing the *sprint backlog*; then each team member takes charge of a subset of stories to design, implement, and test them. Finally, when a sprint is closed, users review the delivered stories to check that they correctly implement the requested functionalities (*sprint review* phase); the approved stories are delivered, while the remaining ones are put back into the product backlog. At this time, the team also analyzes the problems and the solutions taken

so far in order to improve next iterations (e.g., by updating the estimates made so far). Obviously, if new or modified stories arise from user feedback or if estimates are relevantly changed, a new prioritization phase is executed before the next sprint is started.

The *sprint planning problem* can now be informally described as follows. Given a set of user stories and a set of sprints, allocate each story to a sprint so as to meet all the constraints related to the sprint capacity and inter-story dependencies and to fulfill two goals:

1. *Customer satisfaction*. It can be achieved by (i) delivering user stories with higher utility first to increase the user awareness and trust, and by (ii) including affine stories in the same sprint to increase their value for users.
2. *Risk management*. It can be achieved by (i) advancing critical user stories to avoid late side-effects, and by (ii) distributing uncertain stories in different sprints and postponing them to reduce the risk that the sprint delivery is delayed.

The sprint planning problem can be converted into a Generalized Assignment Problem [20] with side constraints, where the knapsacks are the sprints and the items are the user stories. Story points measure the weight of an item, while utility represents its value. Knapsack capacity (i.e., sprint capacity) is measured as the story points that the team can deliver given the sprint duration and the development velocity. The objective function to be maximized is the cumulative utility of the project (goal 1-i), where the utility of each story is increased if some affine stories are included in the same sprint (goal 1-ii) and/or if that story is critical (goal 2-i). Finally, in the formulation of the capacity constraint, the story points of user stories are increased by their uncertainty, which discourages the inclusion of two or more uncertain stories in the same sprint (goal 2-ii). The Generalized Assignment Problem is NP-hard [20].

3. Mathematical formulation

Let $U = \{1, \dots, n\}$ be the index set of the n user stories (or stories for short) to be assigned to sprints. Each story $j \in U$ is associated with its utility u_j , its criticality risk r_j^c , its uncertainty risk r_j^u , and its complexity p_j in story points. Let Y_j be the set of stories affine to story j and a_j be the increment in utility for each affine story assigned to the same sprint (if $Y_j = \emptyset$, we set $a_j = 0$).

Let U^{OR} and U^{AND} be the subsets of stories having dependency type OR and AND, respectively. For each story $j \in U^{OR}$, let D_j^{OR} be the set of stories such that at least one of them must be assigned to the same sprint of story j or to a previous one. Similarly, for each story $j \in U^{AND}$, let D_j^{AND} be the set of stories that must be assigned to the same sprint of story j or to a previous one. Note that each story can be involved in both OR and AND dependencies.

Let $S = \{1, \dots, m\}$ be the index set of the m sprints. Each sprint $i \in S$ has a capacity of p_i^{max} story points.

Let x_{ij} be a binary variable equal to one if the story $j \in U$ is included in sprint $i \in S$, zero otherwise. Let y_{ij} be a non-negative variable equal to the number of stories of Y_j , $j \in U$, included in sprint $i \in S$. The mixed integer linear programming model is the following:

$$(P) \quad z_P = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n u_j(r_j^c x_{ij} + a_j y_{ij}) \quad (1)$$

$$\text{s.t.} \quad \sum_{j=1}^n p_j r_j^u x_{ij} \leq p_i^{max}, \quad i \in S \quad (2)$$

$$\sum_{i=1}^m x_{ij} = 1, \quad j \in U \quad (3)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{OR}} x_{kz} \geq x_{ij}, \quad i \in S, \quad j \in U^{OR} \quad (4)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{AND}} x_{kz} \geq x_{ij} |D_j^{AND}|, \quad i \in S, \quad j \in U^{AND} \quad (5)$$

$$y_{ij} \leq \sum_{k \in Y_j} x_{ik}, \quad i \in S, \quad j \in U \quad (6)$$

$$y_{ij} \leq |Y_j| x_{ij}, \quad i \in S, \quad j \in U \quad (7)$$

$$x_{ij} \in \{0, 1\}, \quad i \in S, \quad j \in U \quad (8)$$

$$y_{ij} \geq 0, \quad i \in S, \quad j \in U \quad (9)$$

The objective function (1) maximizes the cumulative utility function. The utility u_j of story j is increased by its criticality risk r_j^{cr} , thus encouraging an early placement of critical stories, and by the affinity increment a_j for each affine story included in the same sprint. Given story j , the number of affine stories included in sprint i is $y_{ij} = \sum_{k \in Y_j} x_{ik}$, if $x_{ij} = 1$, and $y_{ij} = 0$ otherwise. Since we deal with a maximization problem, constraints (6) and (7) guarantee the correct evaluation of each variable y_{ij} , that does not require an explicit integrality constraint. Constraints (2) ensure that the overall complexity of all the stories assigned to each sprint does not exceed the sprint capacity, while constraints (3) guarantee that each story is assigned to a sprint. Dependencies are imposed by constraints (4) and (5). If a story j has an OR dependency, constraints (4) enable j to be assigned to sprint i only if at least one story in set D_j^{OR} is assigned to a sprint $i' \leq i$. Similarly, if j has an AND dependency, constraints (5) enable j to be assigned to sprint i only if all stories in set D_j^{AND} are assigned to sprints less than or equal to i .

This mathematical formulation can be solved by a general purpose MIP solver. However, to reduce the computing time we also propose some reduction procedures and some valid inequalities. The valid inequalities investigated are the classical *Lifted Cover Inequalities* and some *Dominance Inequalities*.

3.1. Reductions

The reduction procedures try to strengthen the capacity constraints (2) by modifying either the sprint capacities or the weights $pr_j = p_j r_j^{un}$ of the user stories. Similar reductions are used for packing problems in [6,10,11].

3.1.1. Modifying the sprint capacities

If no combination of user stories exactly filling the capacity p_i^{max} of sprint $i \in S$ exists, then there are useless story points that can be removed from the sprint capacity without modifying the optimal solution value. The capacity of a sprint i can be updated by solving the following subset sum problem:

$$p_i^{max} = \max \left\{ \sum_{k \in U} pr_k \xi_k : \sum_{k \in U} pr_k \xi_k \leq p_i^{max}, \xi_k \in \{0, 1\}, j \in U \right\} \quad (10)$$

The subset sum problem can be solved using a simple dynamic programming procedure.

3.1.2. Modifying the weights of stories

It is straightforward to observe that a sprint containing the user story $j \in U$ remains feasible if the weight of j is increased to $pr_j = pr_j + (p_i^{max} - p_{ij}^*)$, where p_{ij}^* is the optimal solution cost of the

following subset sum problem:

$$p_{ij}^* = \max \left\{ p = \sum_{h \in U} pr_h \xi_h : p \leq p_i^{max}, \xi_j = 1, \xi_k \in \{0, 1\}, k \in U \setminus \{j\} \right\} \quad (11)$$

Since we would like to maximize the number of updated weights, we heuristically consider the user stories ordered by non-increasing weights, i.e., $pr_1 \geq pr_2 \geq \dots \geq pr_n$.

3.2. Cover inequalities

The kernel of the proposed mathematical formulation is a Generalized Assignment Problem, therefore the upper bound could be improved by applying valid inequalities for this problem. We have investigated the classic Lifted Cover Inequalities (LCIs) corresponding to the capacity constraints (2), as done in the literature for the Generalized Assignment Problem (see [3]).

We have tried to separate LCIs at each node of the tree search solving the required knapsack problems by a simple dynamic programming procedure. LCIs are usually able to reduce the number of tree search nodes, but, unfortunately, the average computing time to solve each tree node increases too much, i.e., it is on an average about four times the average computing time required by the basic model and in some cases it exceeds the twenty times (see Section 6).

3.3. Dominance inequalities

Given a user story $j \in U$, we define $ur_j = u_j r_j^{cr}$. Two user stories i and j of the same weight, $pr_i = pr_j$, and allocated in two different sprints may be always exchanged. Furthermore, if $ur_i > ur_j$ then it is always more profitable to perform user story i before j (i.e., the objective function is higher). In this case we can say that user story i *dominates* j . Therefore, the basic idea is to add valid inequalities (*Dominance Inequalities*) that forbid solutions where user story j is performed before i . These inequalities are redundant in the original model P, but they could be violated in the LP-relaxation. This idea can be extended considering more than two user stories.

Dominance inequalities can be applied to stories without affinity with other stories (i.e., for which $Y_j = \emptyset$) and only with some combinations of dependencies.

Let $U^D = U^{OR} \cup U^{AND}$ be the set of stories having dependencies and let $D = \bigcup_{j \in U} (D_j^{OR} \cup D_j^{AND})$ be the set of stories on which other stories depend. We define $U' = \{j \in U : Y_j = \emptyset \text{ and } j \notin U^D\}$ and $U'' = \{j \in U : Y_j = \emptyset \text{ and } j \notin D\}$.

3.3.1. Dominance of Type 1

If there exists a pair of user stories $j \in U'$ and $j_1 \in U''$ such that $ur_j > ur_{j_1}$ and $pr_j = pr_{j_1}$, then the following inequalities hold:

$$\sum_{k=1}^{i-1} x_{kj_1} \leq 1 - x_{ij}, \quad \text{for every } i \in S \quad (12)$$

3.3.2. Dominance of Type 2

If there exists a triplet of user stories $j \in U'$ and $j_1, j_2 \in U''$ such that $ur_j > ur_{j_1} + ur_{j_2}$ and $pr_j = pr_{j_1} + pr_{j_2}$, then the following inequalities hold:

$$x_{ij_1} + x_{ij_2} + x_{ij} \leq 2, \quad \text{for every } i, i' \in S \text{ such that } i' < i \quad (13)$$

Similarly, if there exists a triplet of user stories $j \in U''$ and $j_1, j_2 \in U'$ such that $ur_j < ur_{j_1} + ur_{j_2}$ and $pr_j = pr_{j_1} + pr_{j_2}$, then the following inequalities hold:

$$x_{ij_1} + x_{ij_2} + x_{ij} \leq 2, \quad \text{for every } i, i' \in S \text{ such that } i < i' \quad (14)$$

3.3.3. Dominance of Type 3

If there exists a quadruplet of user stories $j \in U'$ and $j_1, j_2, j_3 \in U''$ such that $ur_j > ur_{j_1} + ur_{j_2} + ur_{j_3}$ and $pr_j = pr_{j_1} + pr_{j_2} + pr_{j_3}$, then the following inequalities hold:

$$x_{ij_1} + x_{ij_2} + x_{ij_3} + x_{ij} \leq 3, \quad \text{for every } i, i' \in S \text{ such that } i' < i \quad (15)$$

Similarly, if there exists a quadruplet of user stories $j \in U''$ and $j_1, j_2, j_3 \in U'$ such that $ur_j < ur_{j_1} + ur_{j_2} + ur_{j_3}$ and $pr_j = pr_{j_1} + pr_{j_2} + pr_{j_3}$, then the following inequalities hold:

$$x_{ij_1} + x_{ij_2} + x_{ij_3} + x_{ij} \leq 3, \quad \text{for every } i, i' \in S \text{ such that } i < i' \quad (16)$$

It is quite obvious that dominance inequalities can be easily generalized with respect to a parameter $k \leq n$, if there exists at least a story j such that $pr_j = \sum_{i=1}^k pr_{j_i}$. However, computational results show that there are no benefits in spite of an increasing computational complexity.

4. Greedy and exchange heuristics

In this section we propose two greedy heuristics and a post-optimization procedure based on exchanges.

The first heuristic is based on a simple idea. Following a greedy approach, the procedure starts by optimizing sprint $i=1$ and, then, optimizes the remaining sprints in turn, one at a time, following chronological order. Therefore, at each iteration, the greedy procedure considers a sprint $i \in S$ and assigns to it the stories that maximize the utility by solving the following subproblem:

$$(SP_i) \quad z_{SP_i} = \max \sum_{j=1}^n (m-i+1)u_j(r_j^{cr}x_{ij} + a_jy_{ij}) \quad (17)$$

$$\text{s.t.} \quad \sum_{j=1}^n p_j r_j^{um} x_{ij} \leq p_i^{\max} \quad (18)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{OR}} x_{kz} \geq x_{ij} - |D_j \cap F_i|, \quad j \in U^{OR} \quad (19)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{AND}} x_{kz} \geq x_{ij} |D_j| - |D_j \cap F_i|, \quad j \in U^{AND} \quad (20)$$

$$y_{ij} \leq \sum_{k \in Y_j} x_{ik}, \quad j \in U \quad (21)$$

$$y_{ij} \leq |Y_j| x_{ij}, \quad j \in U \quad (22)$$

$$x_{ij} \in \{0, 1\}, \quad j \in U \setminus F_i \quad (23)$$

$$x_{ij} = 0, \quad j \in F_i \quad (24)$$

$$y_{ij} \geq 0, \quad i \in S, j \in U \quad (25)$$

where F_i represents the stories already assigned to sprints considered in the previous iterations (at the beginning, $F_1 = \emptyset$). Note that the stories F_i cannot be allocated to sprint i (see constraints (24)) and must be considered in dependency constraints (19) and (20).

The greedy heuristic, summarized in Fig. 2, is very fast, as shown in Section 6, but if it must be repeated many times, as in the Lagrangian heuristic described in Section 5, the overall

```

GREEDYHEURISTIC()
1 Set  $F_1 = \emptyset$ ,  $i = 1$ .
2 while ( $F_i \neq U$  and  $i \leq m$ ) do
3   Compute the optimal solution  $x^*$  of subproblem  $SP_i$ .
4   Set  $F_{i+1} = F_i \cup \{j \in U : x_{ij}^* = 1\}$ .
5   Set  $i = i + 1$ .

```

Fig. 2. Algorithm GreedyHeuristic.

computing time can become too large. For example, in Section 6 we show that for some large instances GreedyHeuristic requires more than one second, thus if it is repeated for thousands of iterations, thousands of seconds are wasted just for the greedy. Therefore, we propose a modified greedy heuristic, where the subproblem SP_i is relaxed removing constraints (19)–(22) and the resulting subproblem, called SP'_i , is a knapsack problem which can be efficiently solved by dynamic programming. Unfortunately, the relaxed subproblem has two weaknesses. First, without the linking constraints (21) and (22), variables $\{y_{ij}\}$ are independent. We solve this issue ignoring variables $\{y_{ij}\}$ when the knapsack problem SP'_i is solved and we post-evaluate them using the variables $\{x_{ij}\}$ corresponding to the solution of SP'_i (i.e., we set $y_{ij} = \sum_{k \in Y_j} x_{ik}$, if $x_{ij} = 1$, or $y_{ij} = 0$ otherwise). Second, without constraints (19) and (20), some dependencies can be violated. Given the current sprint i and a user story j having a dependency constraint violated, to recover feasibility, we propose four different strategies:

- Forbid the use of user story j in sprint i (i.e., fix $x_{ij} = 0$) and re-optimize the knapsack problem SP'_i .
- Fix in the solution the user story $j' \in D_j^{OR}$ (or $j' \in D_j^{AND}$, depending on the violated constraint), not included in the current SP'_i solution, that maximizes the ratio $u_j r_j^{cr} / p_j r_j^{um}$. That is, fix $x_{ij'} = 1$ and re-optimize the knapsack problem SP'_i .
- Only for the AND dependency constraints, fix in the solution all the user stories $j' \in D_j^{AND} \setminus F_i$ not included in the current SP'_i solution (i.e., fix $x_{ij'} = 1$) and re-optimize the knapsack problem SP'_i . For the OR dependency constraints apply strategy (ii).
- For every user story j define a coefficient κ_j to increase its profit in every knapsack problems SP'_i (i.e., the profit is multiplied by κ_j). For every user story $j' \in D_j^{OR} \setminus F_i$ (or $j' \in D_j^{AND} \setminus F_i$, depending on the violated constraint) having $x_{ij} = 0$ in the current solution, increase the coefficient κ_j using one of the following rules: (a) $\kappa_j = \rho \times \kappa_j$ or (b) $\kappa_j = \kappa_j \times \kappa_j$. Restart the greedy heuristic from the first sprint, i.e., set $i=1$ and $F_1 = \emptyset$.

Remarkably, the first strategy guarantees the convergence to a feasible solution, whereas the second and the third strategies may give rise to unfeasible knapsack instances, in particular the third one. When a knapsack problem SP'_i does not have a feasible solution we skip to the next greedy heuristic iteration. The fourth strategy could require too many iterations to reach a feasible solution, therefore a maximum number of iterations $MaxIter$ must be set.

The modified greedy procedure, called QuickGreedyHeuristic, is summarized in Fig. 3. In our computational results we set

```

QUICKGREEDYHEURISTIC()
1 Set  $z^* = -\infty$ .
2 foreach Strategy  $s = 1, 2, 3, 4$  do
3   Set  $z' = 0$ ,  $F_1 = \emptyset$ ,  $i = 1$ ,  $Iter = 0$ .
4   while ( $F_i \neq U$  and  $i \leq m$ ) do
5     repeat
6       Compute the optimal solution  $x'$  of knapsack problem  $SP'_i$ .
7       if solution  $x'$  violates some dependency
8         then Apply strategy  $s$ .
9       if  $s = 4$ 
10        then Set  $z' = 0$ ,  $F_1 = \emptyset$ ,  $i = 1$ ,  $Iter = Iter + 1$ .
11    until ( $x'$  is feasible or  $SP'_i$  has not a feasible solution
12        or  $Iter > MaxIter$ )
13    if  $SP'_i$  has not a feasible solution or  $Iter > MaxIter$ 
14      then Set  $z' = -\infty$  and  $i = m + 1$ .
15    else Set  $z' = z' + z_{SP'_i}$ .
16      Set  $F_{i+1} = F_i \cup \{j \in U : x'_{ij} = 1\}$ .
17      Set  $i = i + 1$ .
18  if  $z^* < z'$ 
19    then Set  $z^* = z'$  and  $x^* = x'$ .

```

Fig. 3. Algorithm QuickGreedyHeuristic.

$MaxIter=1000$ and the fourth strategy is applied one time using rule (b) setting $\kappa_j = 1.025$, for every $j \in U$, and two times using rule (a) setting $\kappa_j = 1$, for every $j \in U$, and $\rho=2$ or $\rho=5$. The choice of parameters κ_j and ρ takes into account the trade-off between the time for obtaining a feasible solution and its quality. If some profits increase too quickly we obtain a feasible solution in a few iterations but probably its quality is poor (because we quickly move the corresponding user stories to the first sprints without taking enough care of their real utilities), whereas if profits increase too slowly we need too many iterations for obtaining a feasible solution.

Both algorithms *GreedyHeuristic* and *QuickGreedyHeuristic* may terminate without finding a feasible solution, because all the sprints are considered (i.e., $i > m$) but not all the user stories have been assigned to the available sprints (i.e., $F_i \neq U$). Moreover, a feasible solution provided by *QuickGreedyHeuristic* can be usually further improved, because applying the dependency feasibility recovering strategies can generate optimal solution non-locally.

To improve a feasible solution, a local search based on exchanges can be applied. We propose an exchange heuristic, called *ExchangeHeuristic*, described in Fig. 4. Procedure *ExchangeHeuristic* tries to exchange user stories between two sprints. Namely the procedure tries the following exchanges:

- 1-0: move a story $j \in U$ from sprint i to sprint i' ;
- 1-1: exchange a story $j \in U$ executed in sprint i with a story $j' \in U$ executed in sprint i' ;
- 2-1: exchange two stories $j, j' \in U$ executed in sprint i with a story $j'' \in U$ executed in sprint i' .

An exchange is performed only if it is *feasible* and *profitable*. It is feasible if, after the exchange, the capacity constraints of the corresponding sprints i and i' are still satisfied and dependency constraints are not violated. It is profitable if the overall objective function is increased.

Procedure *ExchangeHeuristic* could be extended adding other more complex exchanges, but in spite of the increasing computational complexity the improvements are usually negligible.

5. A Lagrangian heuristic

The literature is rich with heuristics based on decomposition methods. An excellent introduction to the whole topic of Lagrangian relaxation, and of related heuristics, can be found in [4,7,8].

The Lagrangian relaxation is obtained from model P, described in Section 3, by dualizing constraints (3), (4), (5), (6) and (7) by means

of penalties $\{\lambda_j\}$, $\{\lambda_{ij}^{OR}\}$, $\{\lambda_{ij}^{AND}\}$, $\{\lambda_{ij}^{Y1}\}$, and $\{\lambda_{ij}^{Y2}\}$, respectively. Lagrangian penalties λ_j , $j \in U$, are unconstrained, whereas the remaining penalties are non-positive. The corresponding Lagrangian problem is the following:

$$(LR) \quad z_{LR}(\lambda) = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n (u'_{ij}(\lambda)x_{ij} + u''_{ij}(\lambda)y_{ij}) + \sum_{j=1}^n \lambda_j \quad (26)$$

$$\text{s.t.} \quad \sum_{j=1}^n p_j r_j^{um} x_{ij} \leq p_i^{max}, \quad i \in S \quad (27)$$

$$x_{ij} \in \{0, 1\}, \quad i \in S, j \in U \quad (28)$$

$$0 \leq y_{ij} \leq |Y_j|, \quad i \in S, j \in U \quad (29)$$

where the *penalized utilities* $u'_{ij}(\lambda)$ and $u''_{ij}(\lambda)$ are given by

$$\begin{aligned} u'_{ij}(\lambda) &= u_j r_j^{cr} - \lambda_j + \left(\lambda_{ij}^{OR} - \sum_{k=i}^m \sum_{j' \in \bar{D}_j^{OR}} \lambda_{kj'}^{OR} \right) \\ &\quad + \left(|D_j^{AND}| \lambda_{ij}^{AND} - \sum_{k=i}^m \sum_{j' \in \bar{D}_j^{AND}} \lambda_{kj'}^{AND} \right) - \lambda_{ij}^{Y1} - |Y_j| \lambda_{ij}^{Y2} \\ u''_{ij}(\lambda) &= u_j a_j + \lambda_{ij}^{Y1} + \lambda_{ij}^{Y2} \end{aligned} \quad (30)$$

where $\bar{D}_j^{OR} = \{j' \in U : j \in D_j^{OR}\}$ and $\bar{D}_j^{AND} = \{j' \in U : j \in D_j^{AND}\}$.

The Lagrangian problem LR can be decomposed into 2 m independent subproblems, two for each sprint $i \in S$, as shown in the following:

$$(LR_i^1) \quad z_{LR_i^1}(\lambda) = \max \sum_{j=1}^n u'_{ij}(\lambda)x_{ij} \quad (31)$$

$$\text{s.t.} \quad \sum_{j=1}^n p_j r_j^{um} x_{ij} \leq p_i^{max} \quad (32)$$

$$x_{ij} \in \{0, 1\}, \quad j \in U \quad (33)$$

and

$$(LR_i^2) \quad z_{LR_i^2}(\lambda) = \max \sum_{j=1}^n u''_{ij}(\lambda)y_{ij} \quad (34)$$

$$\text{s.t.} \quad 0 \leq y_{ij} \leq |Y_j|, \quad j \in U \quad (35)$$

Subproblem LR_i^1 is a knapsack problem whereas subproblem LR_i^2 can be easily solved by inspection (i.e., if $u''_{ij} > 0$, $y_{ij} = |Y_j|$, otherwise $y_{ij} = 0$). The overall optimal solution value of the Lagrangian problem LR is given by

$$z_{LR}(\lambda) = \sum_{i=1}^m (m-i+1)(z_{LR_i^1}(\lambda) + z_{LR_i^2}(\lambda)) + \sum_{j=1}^n \lambda_j \quad (36)$$

that is a valid upper bound for the original problem P. In order to find the penalty vector λ^* that minimizes the upper bound $z_{LR}(\lambda)$ we must solve the *Lagrangian Dual* $z_{LR}(\lambda^*) = \min_{\lambda} \{z_{LR}(\lambda)\}$. This can be done heuristically by a subgradient algorithm [24], i.e., an iterative procedure that, at each iteration k , computes a new approximation λ^{k+1} of the Lagrangian multipliers in such a way that, for $k \rightarrow +\infty$, λ^k is an optimal or a near-optimal solution to the corresponding Lagrangian Dual.

Let $(\mathbf{x}, \mathbf{y})$ be the solution of cost $z_{LR}(\lambda)$ obtained at a given iteration by solving the Lagrangian problem LR. The Lagrangian multipliers can be updated as follows:

$$\begin{aligned} \lambda_j &= \lambda_j + \alpha g_j, \quad j \in U \\ \lambda_{ij}^{OR} &= \max\{0, \lambda_{ij}^{OR} + \alpha g_{ij}^{OR}\}, \quad i \in S, j \in U^{OR} \\ \lambda_{ij}^{AND} &= \max\{0, \lambda_{ij}^{AND} + \alpha g_{ij}^{AND}\}, \quad i \in S, j \in U^{AND} \\ \lambda_{ij}^{Y1} &= \max\{0, \lambda_{ij}^{Y1} + \alpha g_{ij}^{Y1}\}, \quad i \in S, j \in U \end{aligned}$$

```

EXCHANGEHEURISTIC()
1  Let x' be the solution to improve.
2  repeat
3    // Apply 1-0 exchanges
4    foreach sprint i = 1, ..., m-1 do
5      foreach sprint i' = i+1, ..., m do
6        foreach j in U such that x'_{ij} = 1 do
7          if moving j from i' to i is feasible and profitable
8            then x'_{ij} = 0 and x'_{ij'} = 1
9    // Apply 1-1 exchanges
10   foreach sprint i = 1, ..., m-1 do
11     foreach sprint i' = i+1, ..., m do
12       foreach j, j' in U such that x'_{ij} = x'_{i'j'} = 1 do
13         if exchanging j and j' is feasible and profitable
14           then x'_{ij} = x'_{i'j'} = 0 and x'_{ij'} = x'_{i'j} = 1
15   // Apply 2-1 exchanges
16   foreach sprint i = 1, ..., m do
17     foreach sprint i' = 1, ..., m do
18       foreach j, j', j'' in U such that x'_{ij} = x'_{i'j'} = x'_{i'j''} = 1 do
19         if exchanging j with j' and j'' is feasible and profitable
20           then x'_{ij} = x'_{i'j'} = x'_{i'j''} = 0 and x'_{ij'} = x'_{i'j} = x'_{i'j''} = 1
21   until (at least one exchange occurs)

```

Fig. 4. Algorithm *ExchangeHeuristic*.

```

LAGRANGIANHEURISTIC()
1  Set  $\lambda = 0$ ,  $z^* = -\infty$ ,  $z_{LR}^* = \infty$ 
2  repeat
3    Compute  $z_{LR}(\lambda)$  solving the Lagrangian problem LR
4    if  $z_{LR}(\lambda) < z_{LR}^*$ 
5      then  $z_{LR}^* = z_{LR}(\lambda)$ 
6    Compute a heuristic solution  $\mathbf{x}'$  with QuickGreedyHeuristic using
7      the penalized utilities and improve  $\mathbf{x}'$  with ExchangeHeuristic
8    Let  $z'$  be the value of the improved solution  $\mathbf{x}'$ 
9    if  $\gamma z^* \leq z'$ 
10     then Compute a heuristic solution  $\mathbf{x}''$  with GreedyHeuristic
11       using the penalized utilities
12     Let  $z''$  be the value of solution  $\mathbf{x}''$ 
13     if  $z^* < z''$ 
14       then  $z^* = z''$  and  $\mathbf{x}^* = \mathbf{x}''$ 
15   if  $z^* < z'$ 
16     then  $z^* = z'$  and  $\mathbf{x}^* = \mathbf{x}'$ 
17   Update penalties  $\lambda$ 
18 until (the subgradient end conditions are satisfied)

```

Fig. 5. Algorithm *LagrangianHeuristic*.

$$\lambda_{ij}^{Y2} = \max\{0, \lambda_{ij}^{Y2} + \alpha g_{ij}^{Y2}\}, \quad i \in S, j \in U \quad (37)$$

where α is the length of the step along the search direction given by the subgradient $\mathbf{g}$ whose components are

$$\begin{aligned}
g_j &= \sum_{i=1}^m x_{ij} - 1, \quad j \in U \\
g_{ij}^{OR} &= \sum_{k=1}^i \sum_{z \in D_j^{OR}} x_{kz} - x_{ij}, \quad i \in S, j \in U^{OR} \\
g_{ij}^{AND} &= \sum_{k=1}^i \sum_{z \in D_j^{AND}} x_{kz} - x_{ij} |D_j^{AND}|, \quad i \in S, j \in U^{AND} \\
g_{ij}^{Y1} &= \sum_{k \in Y_j} x_{ik} - y_{ij}, \quad i \in S, j \in U \\
g_{ij}^{Y2} &= |Y_j| x_{ij} - y_{ij}, \quad i \in S, j \in U
\end{aligned} \quad (38)$$

In our computational experiment $\alpha = \beta(0.1z_{LR}(\lambda)/\|\mathbf{g}\|_2^2)$, where β is initialized with a value that is problem-dependent (in our case, $\beta=3$) and, if after a given number of steps (in our case, 10) the solution value $z_{LR}(\lambda)$ is not improved, then β is reduced (in our case, $\beta=0.85\beta$). The maximum number of iterations is 5000, but if within 50 iterations $z_{LR}(\lambda)$ is not improved by at least 0.01%, the subgradient algorithm is stopped in advance.

The heuristic procedure based on the proposed Lagrangian relaxation is summarized in Fig. 5. At each iteration of the subgradient algorithm a heuristic solution $\mathbf{x}'$ is computed with procedure *QuickGreedyHeuristic* using the penalized utilities computed according to expression (30). The heuristic solution $\mathbf{x}'$ is further improved by procedure *ExchangeHeuristic*. The solution $\mathbf{x}'$ of value z' replaces the best solution found so far $\mathbf{x}^*$ if z' improves z^* (i.e., $z^* < z'$). Moreover, if $\gamma z^* \leq z'$ a new heuristic solution $\mathbf{x}''$ is also computed with the more expensive procedure *GreedyHeuristic*. If we choose $\gamma=1$, we execute *GreedyHeuristic* only if $\mathbf{x}'$ is the best solution found so far, while if $\gamma < 1$, we execute *GreedyHeuristic* if z' has a percentage distance from the best value z^* within $100 \times (1 - \gamma)$. The solution $\mathbf{x}''$ of value z'' replaces the best solution found so far $\mathbf{x}^*$ if z'' improves z^* (i.e., $z^* < z''$). At the end of the *LagrangianHeuristic* algorithm we have a feasible solution of value z^* and an upper bound z_{LR}^* that can be used to evaluate the maximum distance of the heuristic solution found from the optimality.

6. Computational results

The algorithms presented in this paper have been coded in C++ using Microsoft Visual Studio 2010, and run on a workstation equipped with an Intel Xeon X7350 2.94 GHz, 16 Gb of

RAM and operating system Windows Server 2003 64 bit. Ilog CPLEX 12.4 was used as the MIP solver.

We have used datasets coming from both real and synthetic projects. Our real projects have different characteristics and come from different areas; they were carried out by Italian companies that have been successfully adopting agile methods for several years. Their features are summarized in the following:

- PayTV was aimed at developing a data mart for a large company in the area of pay-tvs. The project had an overall duration of 8 months; it included 44 user stories and consisted of 10 sprints with an average duration of 17 days per sprint. 35 dependencies (mainly of AND type) and just one affinity were involved. The development velocity used to run the optimization model was 2.43 story points per day and was empirically determined based on historical data.
- Web was aimed at developing a complex web site based on a Content Management System. Web is larger than PayTV in terms of number of user stories (104 overall); it was organized in 4 sprints of 10 days each, so it had a shorter overall duration (40 days). This difference is due to the lower complexity of the single user stories and to a higher development velocity (6 story points per day). Compared to PayTV, Web includes a small number of chain dependencies (6 overall) and no affinity constraints.

From the effectiveness point of view we remark that all of the optimal plans generated by our algorithms have been judged to be feasible and realistic by team managers, and most of the differences in sprint compositions were evaluated as improvements over the team plan (see [16] for a detailed analysis).

To better stress our algorithms we also generated a set of synthetic projects. Our project generator initially creates user stories by randomly assigning their utility, risk, and complexity. Then, it randomly adds groups of dependencies organized either in chains (one story only depends on one story) or in DAGs (one story depends on several stories). Finally, sets of affine stories are defined. We set the sprint capacity and the development velocity to 45 story points and 3 story points per day, respectively (i.e., each sprint takes 15 days).

Table 1 summarizes the key features of each project: the number n of stories; the maximum number m of sprints; the number n_{aff} of stories involved in at least one affinity; the cardinality of U^{OR} and U^{AND} ; the maximum length l_{max} of groups of dependencies; and the maximum number d_{max} of dependencies involving a single user story. Projects are clustered into five groups: group A contains the real projects, while the projects in groups B and C show a mix of the previous parameters and vary in size, types of dependencies, and presence of affinities. In Group D, the utility of stories is strongly correlated to their complexity (the complexity is always twice the utility); finally, the projects in group E are characterized by stories with high complexity so that each sprint can include at most 5 stories.

The computational experiments are reported in Tables 2–5 that include the following columns:

- z : the value of the best feasible solution found by each algorithm;
- z_{best} : the value of the best feasible solution that was found during the different computational tests by IBM Ilog Cplex or *LagrangianHeuristic*;
- Gap**: the percentage gap between the best feasible solution and the upper bound associated to the best node remaining provided by IBM Ilog Cplex;
- Nodes**: the number of tree nodes generated by IBM Ilog Cplex;
- Cuts**: the number of cuts added by means of the IBM Ilog Cplex *callbacks*; this number includes both constraints (4) and (5), added as *lazy constraints*, as well as Dominance Inequalities (DIs) and LCIs;

Table 1
Problem instances.

Group	Proj. name	n	m	n_{aff}	$ U^{OR} $	$ U^{AND} $	l_{max}	d_{max}
A – real	PayTV	44	12	2	8	27	6	5
	Web	104	6	5	0	4	4	1
B – basic	25Chain-1	25	9	0	0	12	4	1
	25Graph-1	25	8	0	10	1	2	2
	25Affinity-1	25	9	6	5	5	2	2
	50Chain-1	50	12	0	0	20	4	1
	50Graph-1	50	11	0	8	10	2	2
	50Affinity-1	50	12	6	8	9	2	2
	75Chain-1	75	17	0	0	35	5	1
	75Graph-1	75	19	0	13	20	2	2
	75Affinity-1	75	17	6	17	13	2	3
	100Chain-1	100	20	0	0	40	5	1
	100Graph-1	100	23	0	20	14	2	3
	100Affinity-1	100	22	6	16	14	3	4
C – basic	25Chain-2	25	8	0	0	12	2	1
	25Graph-2	25	8	0	3	8	3	2
	25Affinity-2	25	9	6	7	4	3	2
	50Chain-2	50	13	0	0	10	5	1
	50Graph-2	50	13	0	13	8	4	5
	50Affinity-2	50	13	6	13	9	3	3
	75Chain-2	75	17	0	0	36	3	1
	75Graph-2	75	18	0	14	16	2	2
	75Affinity-2	75	18	6	17	13	2	2
	100Chain-2	100	22	0	0	30	5	1
	100Graph-2	100	22	0	12	15	5	7
	100Affinity-2	100	22	6	11	14	3	2
D – correlated	25Chain-3	25	15	0	0	12	4	1
	25Graph-3	25	15	0	5	7	5	4
	25Affinity-3	25	13	6	5	4	2	2
	50Chain-3	50	22	0	0	20	4	1
	50Graph-3	50	22	0	7	9	2	2
	50Affinity-3	50	21	6	9	6	2	2
	75Chain-3	75	31	0	0	36	6	1
	75Graph-3	75	29	0	12	17	2	2
	75Affinity-3	75	33	6	20	7	2	2
	100Chain-3	100	38	0	0	40	8	1
	100Graph-3	100	40	0	16	14	3	3
	100Affinity-3	100	43	6	16	13	2	2
B – few	25Chain-4	25	12	0	5	7	4	1
	25Graph-4	25	13	0	5	6	5	4
	25Affinity-4	25	14	6	3	7	2	2
	50Chain-4	50	21	0	9	11	4	1
	50Graph-4	50	21	0	2	13	2	2
	50Affinity-4	50	21	6	6	8	2	2
	75Chain-4	75	27	0	15	21	6	1
	75Graph-4	75	29	0	12	12	2	4
	75Affinity-4	75	29	6	10	12	2	2
	100Chain-4	100	40	0	24	21	9	1
	100Graph-4	100	40	0	8	17	2	2
	100Affinity-4	100	39	6	18	10	2	2

LGap: the percentage gap between the upper bound z_{LR}^* and the value z^* of the heuristic solution provided by *Lagrangian-Heuristic*, i.e., $LGap = 100 \times (z_{LR}^* - z^*)/z^*$;

BGap: the percentage gap between the value z_{best} of the best feasible solution known and the value z of the feasible solution found by each algorithm, i.e., $BGap = 100 \times (z_{best} - z)/z_{best}$;

Time: the overall computing time in seconds;

B/O: in this column we report the letter “b” if the feasible solution found by the corresponding algorithm is the best for the computational test reported in the table, and the letter “o” if the solution is optimal.

In our computational tests we set a time limit of 600 s for the results reported in Tables 2 and 3, of 60 s for Table 4, and of 10 s for Table 5. When IBM Ilog Cplex does not find a feasible solution for an instance within the given time limit, we report the character

“–” in columns z , *Gap*, and *BGap*. At the bottom of each table we summarize the results reporting the following information:

Num: For column B/O we report the number of times the algorithm provides the best solution value in the computational test reported in the table (i.e., number of b's) and the optimal solution (i.e., number of o's);

Max: The maximum value appearing in the corresponding column;

Average: The average of the values appearing in the corresponding column (for Table 2 we consider only the instances where a feasible solution is found);

Average Feasible: The average of the values appearing in the corresponding column computed only considering the instances where all the algorithms have found a feasible solution in the test reported in the table;

Average Solved: The average of the values appearing in the corresponding column computed only considering the instances where all the algorithms have found an optimal solution.

Table 2 shows the computational results obtained solving with IBM Ilog Cplex the basic model (1)–(9) proposed in Section 3, and adding to this model first the DIs described in Section 3.3, and then also the LCIs described in Section 3.2. When the model is solved, the precedence constraints (4) and (5) are added dynamically as *lazy constraints*. The lazy constraints and the valid inequalities are integrated in IBM Ilog Cplex by means of the *callback functions*. Note that, for the basic model, column *Cuts* reports exactly the number of dynamically generated precedence constraints (4) and (5).

Many instances are not solved to optimality by IBM Ilog Cplex within the given time limit of 600 s (30 out of 50 instances). In particular, for instances “100Chain-3” and “100Chain-4” IBM Ilog Cplex cannot find a feasible solution. When we add the DIs and LCIs to the basic model, the results do not improve on average; on the contrary, often IBM Ilog Cplex generates worse solutions and only for some instances the results are strictly improved (i.e., the value z is strictly better than the ones provided by the other versions). The basic model provides the best solution for 36 instances and the strictly best solution for 14 instances. On the other hand, the basic model with DIs provides the best solution for 28 instances and the strictly best solution only for 6 instances (e.g., “75Chain-2”, “50Affinity-3”, “75Chain-4”, etc.), while the basic model with DIs and LCIs provides the best solution for 27 instances and the strictly best solution only for 5 instances (e.g., “75Chain-1”, “100Graph-2”, “100Affinity-2”, etc.). The three versions solve to optimality the same 20 instances.

These results show that DIs and LCIs are usually able to reduce the number of tree nodes, but on an average the cost for separating the inequalities and solving the increased model are not repayed. Row *Average Solved* provides a fair comparison on the number of tree nodes and the corresponding computing times. The basic model with DIs and LCIs reduces the average number of tree nodes from 45876.4 to 20047.1, but increases the average computing time from 23.40 to 54.85 s. On the contrary, the basic model with DIs increases the average number of tree nodes, because probably they induce an increasing number of fractional variables.

Not reported in our computational results, the basic model with only LCIs (without DIs) is on an average outperformed by the basic model with both DIs and LCIs and the contribution of the reduction procedures is negligible. Probably, as in cutting problems, reduction

Table 2

Results obtained solving the basic model with IBM Ilog Cplex and adding valid inequalities.

Name	Basic model						Basic model + DIs						Basic model + DIs + LCIs					
	z	Gap	Nodes	Cuts	Time	B/O	z	Gap	Nodes	Cuts	Time	B/O	z	Gap	Nodes	Cuts	Time	B/O
PayTV	93,330.0	0.00	130,908	238	50.21	b,o	93,330.0	0.00	130,908	238	50.71	b,o	93,330.0	0.00	41,573	3384	96.03	b,o
Web	32,683.6	0.00	0	2	0.30	b,o	32,683.6	0.00	0	2	0.30	b,o	32,683.6	0.00	0	3	0.30	b,o
25Chain-1	16,627.2	0.00	3747	52	0.80	b,o	16,627.2	0.00	2989	56	0.72	b,o	16,627.2	0.00	2790	450	1.15	b,o
25Graph-1	13,515.1	0.00	601	9	0.19	b,o	13,515.1	0.00	601	9	0.19	b,o	13,515.1	0.00	528	93	0.20	b,o
25Affinity-1	18,523.6	0.00	1223	21	0.87	b,o	18,523.6	0.00	1223	21	0.89	b,o	18,523.6	0.00	875	136	1.01	b,o
50Chain-1	41,244.8	0.05	2,284,200	150	600.26	b	41,206.6	0.22	2,180,919	291	600.94		41,244.8	0.13	129,462	7224	600.06	b
50Graph-1	34,686.0	0.00	55,654	43	16.41	b,o	34,686.0	0.00	208,673	103	57.63	b,o	34,686.0	0.00	176,231	3920	300.57	b,o
50Affinity-1	40,545.1	0.00	3486	46	2.50	b,o	40,545.1	0.00	2993	68	2.33	b,o	40,545.1	0.00	699	244	1.11	b,o
75Chain-1	88,713.3	0.91	874,300	369	600.22		87,879.6	1.96	616,962	1202	600.47		88,754.5	1.24	43,100	11,288	600.35	b
75Graph-1	92,959.2	0.23	873,145	188	600.27	b	92,911.4	0.32	548,418	1517	600.75		92,952.8	0.29	42,113	13,028	600.39	
75Affinity-1	76,076.2	0.00	165,046	80	193.37	b,o	76,076.2	0.00	162,370	359	191.02	b,o	76,076.2	0.00	88,979	8904	508.81	b,o
100Chain-1	136,240.7	0.96	608,866	529	600.42	b	135,965.9	1.40	306,300	2758	600.68		136,159.3	1.65	34,401	13,408	600.29	
100Graph-1	149,517.0	0.23	647,382	173	600.40	b	149,453.7	0.28	277,600	2600	600.99		149,462.2	0.28	50,300	12,315	600.63	
100Affinity-1	136,008.3	0.25	281,057	138	600.26	b	135,975.9	0.29	307,700	2222	600.25		135,965.8	0.31	54,300	11,950	600.20	
25Chain-2	13,214.8	0.00	212	30	0.17	b,o	13,214.8	0.00	230	29	0.19	b,o	13,214.8	0.00	128	68	0.16	b,o
25Graph-2	17,202.6	0.00	199	19	0.17	b,o	17,202.6	0.00	223	23	0.17	b,o	17,202.6	0.00	355	83	0.20	b,o
25Affinity-2	13,007.4	0.00	199	26	0.22	b,o	13,007.4	0.00	199	30	0.20	b,o	13,007.4	0.00	128	94	0.20	b,o
50Chain-2	46,629.1	0.00	29,211	71	10.84	b,o	46,629.1	0.00	109,674	170	33.10	b,o	46,629.1	0.00	31,345	2818	82.32	b,o
50Graph-2	37,699.9	0.00	375,337	103	116.81	b,o	37,699.9	0.00	373,877	238	126.02	b,o	37,699.9	0.00	14,907	2868	36.78	b,o
50Affinity-2	46,153.1	0.00	34,993	112	35.09	b,o	46,153.1	0.00	78,374	230	73.60	b,o	46,153.1	0.00	25,176	2509	46.66	b,o
75Chain-2	78,754.0	1.62	829,001	476	600.42		79,260.6	0.97	593,600	831	600.38	b	79,175.2	1.48	40,277	12,123	600.35	
75Graph-2	72,519.7	0.24	1,026,580	116	600.34		72,569.1	0.23	814,000	1069	600.85	b	72,525.0	0.29	50,721	13,383	600.59	
75Affinity-2	84,372.4	0.06	314,700	133	600.15	b	84,284.1	0.17	342,500	461	600.22		84,274.4	0.28	51,029	13,626	600.11	
100Chain-2	134,485.9	0.26	654,377	419	600.40	b	–	–	272,000	3873	601.25		133,814.5	0.97	34,311	13,574	600.50	
100Graph-2	134,975.2	0.31	644,156	170	600.46		134,866.8	0.41	296,500	3038	601.15		135,023.3	0.28	49,300	12,415	600.71	b
100Affinity-2	136,435.5	0.25	301,800	144	600.29		136,447.8	0.25	329,765	4076	600.26		136,471.8	0.23	75,570	11,976	600.21	b
25Chain-3	2855.2	0.00	26,378	122	12.79	b,o	2855.2	0.00	48,121	128	21.59	b,o	2855.2	0.00	3940	467	5.21	b,o
25Graph-3	1935.7	0.00	59,936	89	13.96	b,o	1935.7	0.00	49,015	144	11.89	b,o	1935.7	0.00	4243	526	3.87	b,o
25Affinity-3	2002.3	0.00	4538	45	5.41	b,o	2002.3	0.00	3217	49	4.01	b,o	2002.3	0.00	2132	275	3.93	b,o
50Chain-3	6199.7	2.21	374,400	318	600.20	b	6156.4	3.13	228,157	1525	600.48		6192.1	2.51	42,696	5807	600.31	
50Graph-3	5944.0	1.06	936,113	254	600.29		5933.4	1.39	333,380	3733	601.12		5944.3	1.13	60,300	6905	600.52	b
50Affinity-3	5224.9	0.81	169,000	150	600.28		5229.6	0.73	276,128	1440	600.18	b	5220.4	1.05	64,117	6808	600.13	
75Chain-3	12,206.2	2.71	274,600	901	600.35	b	1,2056.2	4.08	175,200	2308	600.51		12,156.2	3.64	21,386	10,584	600.29	
75Graph-3	11,376.9	1.05	744,845	566	600.60	b	11,365.3	1.18	198,676	5382	600.98		11,355.6	1.28	31,400	9151	600.43	
75Affinity-3	14,415.6	1.92	91,417	280	600.19	b	14,366.6	2.32	88,000	3926	600.12		–	–	28,150	9459	600.09	
100Chain-3	–	–	188,586	1247	600.65		–	–	90,828	5960	600.70		–	–	9591	8884	600.28	
100Graph-3	19,952.5	0.80	431,293	329	600.51	b	–	–	96,716	6466	601.03		–	–	20,170	13,436	600.46	
100Affinity-3	25,412.8	2.96	41,960	712	600.31	b	–	–	39,039	1620	600.57		–	–	16,349	6570	600.53	
25Chain-4	18,240.4	0.00	3230	94	1.37	b,o	18,240.4	0.00	4984	98	1.79	b,o	18,240.4	0.00	1381	554	1.69	b,o
25Graph-4	2,0561.7	0.00	21,922	79	6.49	b,o	20,561.7	0.00	17,496	78	5.41	b,o	20,561.7	0.00	4500	556	4.35	b,o
25Affinity-4	20,308.8	0.00	708	57	1.81	b,o	20,308.8	0.00	699	57	1.81	b,o	20,308.8	0.00	1031	171	2.39	b,o
50Chain-4	60,331.1	1.59	868,901	295	600.26		60,451.9	1.55	625,123	936	600.62		60,485.2	1.69	52,008	5794	600.40	b
50Graph-4	66,335.3	0.75	936,300	179	600.32		66,394.0	0.83	781,206	594	600.60	b	66,394.0	0.61	97,300	4458	600.51	b
50Affinity-4	60,073.0	0.04	218,987	59	600.09	b	60,073.0	0.09	200,625	236	600.23	b	60,006.5	0.31	94,500	4147	600.16	
75Chain-4	110,688.1	5.86	487,300	774	600.67		113,571.4	3.35	328,700	1247	600.56	b	111,994.3	5.25	26,277	11,272	600.35	
75Graph-4	130,342.6	1.26	354,591	229	600.32		130,590.7	1.12	326,200	1501	600.79	b	130,230.8	1.42	41,529	8534	600.50	
75Affinity-4	124,913.5	1.22	91,800	230	600.14	b	124,825.3	1.15	112,200	918	600.18		124,852.5	1.30	40,300	9515	600.10	
100Chain-4	–	–	145,225	1346	600.46		–	–	133,782	1952	600.63		–	–	13,034	11,503	600.28	
100Graph-4	246,939.9	0.86	221,700	278	600.47	b	246,754.3	0.98	158,700	2655	600.90		–	–	26,140	11,014	600.45	
100Affinity-4	237,587.9	0.92	102,582	282	600.40		237,691.5	0.91	91,800	2236	600.25	b	23,7406.5	1.08	28,851	10,859	600.15	
Average/num		0.65	338,733.8	255.4	369.60	36,20		0.65	247,331.8	1414.1	372.04	28,20		0.65	35,398.5	6582.7	382.15	27,20
Average feasible		0.57	352,607.8	189.8	332.03			0.60	267,151.7	1029.1	334.81			0.64	37,725.1	5923.1	346.62	
Average solved		0.00	45,876.4	66.9	23.49			0.00	59,793.3	106.5	29.18			0.00	20,047.1	1406.2	54.85	

Table 3
Comparison among IBM Ilog Cplex, GreedyHeuristic, and LagrangianHeuristic.

Name	z_{best}	IBM Ilog Cplex					GreedyHeuristic				LagrangianHeuristic ($\gamma = 1$)					LagrangianHeuristic ($\gamma = 0.995$)				
		z	Gap	BGap	Time	B/O	z	BGap	Time	B/O	z	LGap	BGap	Time	B/O	z	LGap	BGap	Time	B/O
PayTV	93,330.0	93,330.0	0.00	0.00	50.21	b,o	77,994.0	16.43	0.12		92,646.0	6.01	0.73	19.85		92,646.0	6.01	0.73	25.42	
Web	32,683.6	32,683.6	0.00	0.00	0.30	b,o	32,572.9	0.34	0.03		32,683.6	0.41	0.00	99.57	b,o	32,683.6	0.41	0.00	99.66	b,o
25Chain-1	16,627.2	16,627.2	0.00	0.00	0.80	b,o	16,454.7	1.04	0.28		16,613.6	1.41	0.08	0.53		16,617.1	1.39	0.06	2.79	
25Graph-1	13,515.1	13,515.1	0.00	0.00	0.19	b,o	13,514.7	0.00	0.05		13,515.1	0.95	0.00	0.17	b,o	13,515.1	0.95	0.00	1.34	b,o
25Affinity-1	18,523.6	18,523.6	0.00	0.00	0.87	b,o	18,171.0	1.90	0.06		18,505.1	1.98	0.10	0.83		18,505.1	1.98	0.10	3.90	
50Chain-1	41,244.8	41,244.8	0.05	0.00	600.26	b	40,992.8	0.61	0.14		41,103.9	1.79	0.34	4.52		41,103.9	1.79	0.34	12.92	
50Graph-1	34,686.0	34,686.0	0.00	0.00	16.41	b,o	34,686.0	0.00	0.08	b,o	34,686.0	2.20	0.00	1.70	b,o	34,686.0	2.20	0.00	3.98	b,o
50Affinity-1	40,545.1	40,545.1	0.00	0.00	2.50	b,o	40,373.3	0.42	0.08		40,545.1	0.19	0.00	6.38	b,o	40,545.1	0.19	0.00	14.63	b,o
75Chain-1	88,754.5	88,713.3	0.91	0.05	600.22	b	88,470.2	0.32	0.27		88,470.2	3.20	0.32	46.97		88,522.0	3.14	0.26	49.39	
75Graph-1	92,959.2	92,959.2	0.23	0.00	600.27	b	92,595.2	0.39	0.16		92,844.1	1.59	0.12	63.99		92,892.8	1.53	0.07	99.26	
75Affinity-1	76,076.2	76,076.2	0.00	0.00	193.37	b,o	75,757.2	0.42	0.25		75,876.8	0.55	0.26	35.63		75,897.9	0.52	0.23	62.14	
100Chain-1	136,240.7	136,240.7	0.96	0.00	600.42	b	135,991.8	0.18	0.69		135,991.8	3.09	0.18	73.67		135,991.8	3.09	0.18	73.82	
100Graph-1	149,517.0	149,517.0	0.23	0.00	600.40	b	148,993.6	0.35	0.28		149,452.0	1.46	0.04	123.58		149,452.0	1.46	0.04	162.57	
100Affinity-1	136,008.3	136,008.3	0.25	0.00	600.26	b	135,470.9	0.40	0.38		135,910.4	1.31	0.07	158.93		135,910.4	1.31	0.07	221.44	
25Chain-2	13,214.8	13,214.8	0.00	0.00	0.17	b,o	13,214.8	0.00	0.05	b,o	13,214.8	0.43	0.00	0.58	b,o	13,214.8	0.43	0.00	2.51	b,o
25Graph-2	17,202.6	17,202.6	0.00	0.00	0.17	b,o	17,202.6	0.00	0.03	b,o	17,202.6	1.09	0.00	0.34	b,o	17,202.6	1.09	0.00	2.12	b,o
25Affinity-2	13,007.4	13,007.4	0.00	0.00	0.22	b,o	12,700.3	2.36	0.05		13,007.4	1.86	0.00	0.75	b,o	13,007.4	1.86	0.00	4.62	b,o
50Chain-2	46,629.1	46,629.1	0.00	0.00	10.84	b,o	46,500.7	0.28	0.09		46,545.8	0.85	0.18	3.00		46,545.8	0.85	0.18	7.47	
50Graph-2	37,699.9	37,699.9	0.00	0.00	116.81	b,o	37,693.0	0.02	0.11		37,693.0	3.15	0.02	7.74		37,693.0	3.15	0.02	8.11	
50Affinity-2	46,153.1	46,153.1	0.00	0.00	35.09	b,o	45,766.6	0.84	0.19		45,995.6	2.54	0.34	7.54		45,995.6	2.54	0.34	10.87	
75Chain-2	79,260.6	78,754.0	1.62	0.64	600.42		78,816.4	0.56	0.62	b	78,816.4	4.50	0.56	29.38	b	78,816.4	4.50	0.56	30.55	b
75Graph-2	72,569.1	72,519.7	0.24	0.07	600.34	b	72,408.5	0.22	0.16		72,408.5	5.54	0.22	39.84		72,408.5	5.54	0.22	44.27	
75Affinity-2	84,372.4	84,372.4	0.06	0.00	600.15	b	83,417.6	1.13	0.23		83,758.7	4.90	0.73	68.36		83,758.7	4.90	0.73	80.67	
100Chain-2	134,485.9	134,485.9	0.26	0.00	600.40	b	134,217.7	0.20	0.86		134,217.7	1.25	0.20	49.30		134,217.7	1.25	0.20	56.18	
100Graph-2	135,023.3	134,975.2	0.31	0.04	600.46	b	134,871.6	0.11	0.31		134,871.6	1.03	0.11	124.69		134,871.6	1.03	0.11	181.13	
100Affinity-2	136,471.8	136,435.5	0.25	0.03	600.29	b	136,171.7	0.22	0.47		136,329.0	0.76	0.10	106.38		136,329.0	0.76	0.10	171.79	
25Chain-3	2855.2	2855.2	0.00	0.00	12.79	b,o	2821.9	1.17	0.13		2855.2	0.67	0.00	0.72	b,o	2855.2	0.67	0.00	4.59	b,o
25Graph-3	1935.7	1935.6	0.00	0.01	13.96	b,o	1862.2	3.80	0.11		1919.4	5.03	0.84	0.50		1919.4	5.03	0.84	3.09	
25Affinity-3	2002.3	2002.3	0.00	0.00	5.41	b,o	1976.3	1.30	0.13		1994.9	2.86	0.37	0.89		1994.9	2.86	0.37	1.45	
50Chain-3	6221.7	6199.7	2.21	0.35	600.20	b	6117.0	1.68	0.62		6167.1	4.07	0.88	7.05		6167.1	4.07	0.88	10.36	
50Graph-3	5944.3	5944.0	1.06	0.01	600.29	b	5918.4	0.44	0.75		5918.4	2.53	0.44	4.71		5918.4	2.53	0.44	8.05	
50Affinity-3	5229.6	5224.9	0.81	0.09	600.28	b	5149.2	1.54	0.44		5210.2	3.66	0.37	8.14		5210.2	3.66	0.37	23.76	
75Chain-3	12,206.2	12,206.2	2.71	0.00	600.35	b	11,830.6	3.08	1.39		12,125.9	6.32	0.66	47.24		12,125.9	6.32	0.66	75.10	
75Graph-3	11,376.9	11,376.9	1.05	0.00	600.60	b	11,305.3	0.63	1.83		11,375.4	8.18	0.01	7.30		11,375.4	8.18	0.01	46.04	
75Affinity-3	14,436.8	14,415.6	1.92	0.15	600.19		14,226.8	1.45	1.17		14,436.8	4.00	0.00	13.00	b	14,436.8	4.00	0.00	72.61	b
100Chain-3	19,337.9	–	–	–	600.65		19,336.0	0.01	2.40		19,337.9	5.15	0.00	66.10	b	19,337.9	5.15	0.00	73.01	b
100Graph-3	19,952.5	19,952.5	0.80	0.00	600.51	b	19,859.0	0.47	2.40		19,925.1	3.30	0.14	85.24		19,925.1	3.30	0.14	280.38	
100Affinity-3	25,412.8	25,412.8	2.96	0.00	600.31	b	24,967.7	1.75	2.61		25,371.2	5.60	0.16	53.35		25,371.2	5.60	0.16	74.46	
25Chain-4	18,240.4	18,240.4	0.00	0.00	1.37	b,o	17,500.5	4.06	0.16		18,240.4	3.43	0.00	1.97	b,o	18,240.4	3.43	0.00	2.43	b,o
25Graph-4	20,561.7	20,561.7	0.00	0.00	6.49	b,o	20,237.9	1.57	0.16		20,551.4	5.37	0.05	0.89		20,551.4	5.37	0.05	3.28	
25Affinity-4	20,308.8	20,308.8	0.00	0.00	1.81	b,o	18,820.9	7.33	0.19		20,286.8	6.57	0.11	0.52		20,286.8	6.57	0.11	3.25	
50Chain-4	60,490.4	60,331.1	1.59	0.26	600.26	b	59,741.3	1.24	0.47		60,033.9	4.67	0.75	9.77		60,033.9	4.67	0.75	18.91	
50Graph-4	66,394.0	66,335.3	0.75	0.09	600.32	b	66,166.2	0.34	0.83		66,166.2	4.11	0.34	7.97		66,166.2	4.11	0.34	10.78	
50Affinity-4	60,073.0	60,073.0	0.04	0.00	600.09	b	59,856.4	0.36	0.52		59,912.3	5.12	0.27	13.66		59,912.3	5.12	0.27	65.02	
75Chain-4	113,571.4	110,688.1	5.86	2.54	600.67		111,538.6	1.79	1.34		112,185.6	8.52	1.22	67.13		112,188.4	8.51	1.22	91.09	b
75Graph-4	130,590.7	130,342.6	1.26	0.19	600.32	b	128,400.2	1.68	1.39		129,963.0	5.11	0.48	23.82		129,963.0	5.11	0.48	57.67	
75Affinity-4	124,913.5	124,913.5	1.22	0.00	600.14	b	123,198.2	1.37	1.29		124,196.3	5.54	0.57	31.48		124,196.3	5.54	0.57	62.93	
100Chain-4	216,381.8	–	–	–	600.46		212,681.5	1.71	2.96		216,381.8	9.06	0.00	202.88	b	216,381.8	9.06	0.00	231.12	b
100Graph-4	246,939.9	246,939.9	0.86	0.00	600.47	b	246,505.4	0.18	1.97		246,511.5	4.37	0.17	53.60		246,511.5	4.37	0.17	198.41	
100Affinity-4	237,691.5	237,587.9	0.92	0.04	600.40	b	234,254.3	1.45	2.29		236,779.7	5.84	0.38	53.02		236,779.7	5.84	0.38	131.12	
Max/num			–	–	600.67	45,20		16.43	2.96	4,3		9.06	1.22	202.88	13,9		9.06	1.22	280.38	14,9
Average			–	–	369.60			1.38	0.67			3.46	0.26	36.70			3.46	0.26	59.65	
Average feasible			0.65	0.09	359.98			1.40	0.59			3.31	0.27	32.63			3.31	0.27	55.80	

Table 4Comparison between IBM Ilog Cplex and *LagrangianHeuristic* setting a time limit of 60 s.

Name	z_{best}	IBM Ilog Cplex					LagrangianHeuristic			
		z	Gap	BGap	Time	B/O	z	BGap	Time	B/O
PayTV	93,330.0	93,330.0	0.00	0.00	51.01	b,o	92,646.0	0.73	20.34	
Web	32,683.6	32,683.6	0.00	0.00	0.30	b,o	32,683.6	0.00	60.31	b,o
25Chain-1	16,627.2	16,627.2	0.00	0.00	0.80	b,o	16,613.6	0.08	0.52	
25Graph-1	13,515.1	13,515.1	0.00	0.00	0.19	b,o	13,515.1	0.00	0.17	b,o
25Affinity-1	18,523.6	18,523.6	0.00	0.00	0.87	b,o	18,505.1	0.10	0.86	
50Chain-1	41,244.8	41,182.3	0.46	0.15	60.14	b	41,103.9	0.34	4.49	
50Graph-1	34,686.0	34,686.0	0.00	0.00	16.60	b,o	34,686.0	0.00	1.72	b,o
50Affinity-1	40,545.1	40,545.1	0.00	0.00	2.48	b,o	40,545.1	0.00	6.58	b,o
75Chain-1	88,754.5	84,711.8	6.34	4.55	60.08		88,470.2	0.32	47.49	b
75Graph-1	92,959.2	92,844.7	0.39	0.12	60.11	b	92,844.1	0.12	60.06	
75Affinity-1	76,076.2	76,076.2	0.03	0.00	60.06	b,o	75,876.8	0.26	35.85	
100Chain-1	136,240.7	–	–	–	60.22		135,991.8	0.18	60.11	b
100Graph-1	149,517.0	149,350.9	0.35	0.11	60.20		149,452.0	0.04	60.45	b
100Affinity-1	136,008.3	135,934.8	0.33	0.05	60.15	b	135,910.4	0.07	60.40	
25Chain-2	13,214.8	13,214.8	0.00	0.00	0.16	b,o	13,214.8	0.00	0.59	b,o
25Graph-2	17,202.6	17,202.6	0.00	0.00	0.16	b,o	17,202.6	0.00	0.34	b,o
25Affinity-2	13,007.4	13,007.4	0.00	0.00	0.20	b,o	13,007.4	0.00	0.75	b,o
50Chain-2	46,629.1	46,629.1	0.00	0.00	10.70	b,o	46,545.8	0.18	2.96	
50Graph-2	37,699.9	37,699.9	0.06	0.00	60.15	b,o	37,693.0	0.02	7.84	
50Affinity-2	46,153.1	46,153.1	0.00	0.00	34.67	b,o	45,995.6	0.34	7.65	
75Chain-2	79,260.6	72,821.8	10.75	8.12	60.10		78,816.4	0.56	29.25	b
75Graph-2	72,569.1	72,515.2	0.28	0.07	60.11	b	72,408.5	0.22	39.86	
75Affinity-2	84,372.4	84,198.7	0.37	0.21	60.08	b	83,758.7	0.73	60.09	
100Chain-2	134,485.9	133,767.7	0.94	0.53	60.17		134,217.7	0.20	49.66	b
100Graph-2	135,023.3	134,900.3	0.37	0.09	60.22	b	134,871.6	0.11	60.67	
100Affinity-2	136,471.8	136,355.8	0.32	0.08	60.19	b	136,329.0	0.10	60.45	
25Chain-3	2855.2	2855.2	0.00	0.00	12.40	b,o	2855.2	0.00	0.73	b,o
25Graph-3	1935.7	1935.6	0.00	0.01	13.65	b	1919.4	0.84	0.48	
25Affinity-3	2002.3	2002.3	0.00	0.00	5.20	b,o	1994.9	0.37	0.87	
50Chain-3	6221.7	6221.7	2.07	0.00	60.06	b	6167.1	0.88	6.88	
50Graph-3	5944.3	–	–	–	60.08		5918.4	0.44	4.55	b
50Affinity-3	5229.6	5194.6	1.78	0.67	60.08		5210.2	0.37	8.10	b
75Chain-3	12,206.2	–	–	–	60.06		12,125.9	0.66	46.75	b
75Graph-3	11,376.9	11,316.4	1.62	0.53	60.12		11,375.4	0.01	6.46	b
75Affinity-3	14,436.8	14,290.4	2.98	1.01	60.09		14,436.8	0.00	12.17	b
100Chain-3	19,337.9	–	–	–	60.23		19,337.9	0.00	60.23	b
100Graph-3	19,952.5	19,897.1	1.10	0.28	60.22		19,925.1	0.14	60.08	b
100Affinity-3	25,412.8	–	–	–	60.22		25,371.2	0.16	53.01	b
25Chain-4	18,240.4	18,240.4	0.00	0.00	1.25	b,o	18,240.4	0.00	1.89	b,o
25Graph-4	20,561.7	20,561.7	0.00	0.00	6.33	b,o	20,551.4	0.05	0.83	
25Affinity-4	20,308.8	20,308.8	0.00	0.00	1.73	b,o	20,286.8	0.11	0.44	
50Chain-4	60,490.4	60,490.4	1.74	0.00	60.08	b	60,033.9	0.75	9.20	
50Graph-4	66,394.0	66,262.9	1.00	0.20	60.09	b	66,166.2	0.34	7.47	
50Affinity-4	60,073.0	59,890.1	0.63	0.30	60.06		59,912.3	0.27	13.01	b
75Chain-4	113,571.4	111,280.0	5.85	2.02	60.08		112,185.6	1.22	60.09	b
75Graph-4	130,590.7	129,958.9	1.74	0.48	60.09		129,963.0	0.48	23.01	b
75Affinity-4	124,913.5	124,616.6	1.64	0.24	60.06	b	124,196.3	0.57	31.17	
100Chain-4	216,381.8	–	–	–	60.19		216,381.8	0.00	60.22	b
100Graph-4	246,939.9	244,866.7	1.85	0.84	60.15		246,511.5	0.17	52.04	b
100Affinity-4	237,691.5	236,642.0	1.45	0.44	60.14		236,779.7	0.38	51.85	b
Max/num			–	–	60.23	31,19		1.22	60.67	28,9
Average			–	–	41.66			0.26	26.22	
Average feasible			1.06	0.48	39.13			0.26	23.32	

procedures are effective only for those instances where only a few user stories can be executed at each sprint.

Table 3 provides a comparison among IBM Ilog Cplex applied to the basic model, procedure *GreedyHeuristic* presented in Section 4, and *LagrangianHeuristic* presented in Section 5, setting the time limit equal to 600 s. For *LagrangianHeuristic* we perform two computational tests with two different settings of parameter γ . In the first setting it is $\gamma=1$, therefore at each subgradient iteration the more expensive procedure *GreedyHeuristic* is performed only if *QuickGreedyHeuristic* and *ExchangeHeuristic* provide the best feasible solution computed so far. In the second setting it is $\gamma=0.995$, therefore *GreedyHeuristic* is performed if *QuickGreedyHeuristic* and *ExchangeHeuristic* provide a feasible solution whose value is at least 99.5% the current best solution value.

Procedure *GreedyHeuristic* can solve each instance very quickly, as the average computing time is well under one second, but sometimes it yields unsatisfactory solutions. For example, for instances “PayTV” and “25Affinity-4” the value z of the solution is 16.43% and 7.33% worse than the value z_{best} of the best solution known, respectively. The average gap between the value of the heuristic solution found and the value z_{best} is greater than 1.3%.

LagrangianHeuristic computes solutions of better quality than *GreedyHeuristic*, where the maximum and the average gap between the value of the heuristic solution found and the best solution value known z_{best} is 1.2% and 0.27%, respectively, for both versions with $\gamma=1$ and $\gamma=0.995$. The version with $\gamma=0.995$

Table 5Comparison between IBM Ilog Cplex and *LagrangianHeuristic* setting a time limit of 10 s.

Name	z_{best}	IBM Ilog Cplex					LagrangianHeuristic			
		z	Gap	BGap	Time	B/O	z	BGap	Time	B/O
PayTV	93,330.0	92,781.0	1.40	0.59	10.02	b	92,066.0	1.35	10.08	
Web	32,683.6	32,683.6	0.00	0.00	0.30	b,o	32,683.6	0.00	10.75	b,o
25Chain-1	16,627.2	16,627.2	0.00	0.00	0.81	b,o	16,613.6	0.08	0.52	
25Graph-1	13,515.1	13,515.1	0.00	0.00	0.19	b,o	13,515.1	0.00	0.17	b,o
25Affinity-1	18,523.6	18,523.6	0.00	0.00	0.87	b,o	18,505.1	0.10	0.83	
50Chain-1	41,244.8	41,164.9	0.63	0.19	10.03	b	41,103.9	0.34	4.48	
50Graph-1	34,686.0	34,686.0	0.05	0.00	10.03	b,o	34,686.0	0.00	1.73	b,o
50Affinity-1	40,545.1	40,545.1	0.00	0.00	2.45	b,o	40,545.1	0.00	6.40	b,o
75Chain-1	88,754.5	84,711.8	6.74	4.55	10.06		88,470.2	0.32	10.08	b
75Graph-1	92,959.2	92,753.6	0.58	0.22	10.06	b	92,679.3	0.30	10.28	
75Affinity-1	76,076.2	75,958.6	0.25	0.15	10.03	b	75,841.3	0.31	10.30	
100Chain-1	136,240.7	–	–	–	10.19		135,991.8	0.18	10.22	b
100Graph-1	149,517.0	149,312.2	0.48	0.14	10.17	b	148,993.6	0.35	10.23	
100Affinity-1	136,008.3	135,727.6	0.49	0.21	10.14	b	135,574.6	0.32	10.56	
25Chain-2	13,214.8	13,214.8	0.00	0.00	0.16	b,o	13,214.8	0.00	0.58	b,o
25Graph-2	17,202.6	17,202.6	0.00	0.00	0.16	b,o	17,202.6	0.00	0.33	b,o
25Affinity-2	13,007.4	13,007.4	0.00	0.00	0.19	b,o	13,007.4	0.00	0.78	b,o
50Chain-2	46,629.1	46,629.1	0.02	0.00	10.02	b,o	46,545.8	0.18	2.96	
50Graph-2	37,699.9	37,669.9	0.26	0.08	10.05		37,693.0	0.02	7.72	b
50Affinity-2	46,153.1	46,143.6	0.28	0.02	10.02	b	45,995.6	0.34	7.52	
75Chain-2	79,260.6	–	–	–	10.05		78,816.4	0.56	10.08	b
75Graph-2	72,569.1	72,478.0	0.37	0.13	10.06	b	72,408.5	0.22	10.20	
75Affinity-2	84,372.4	84,047.4	0.61	0.39	10.06	b	83,741.6	0.75	10.06	
100Chain-2	134,485.9	132,748.5	1.85	1.29	10.12		134,217.7	0.20	10.05	b
100Graph-2	135,023.3	134,632.3	0.58	0.29	10.15		134,871.6	0.11	10.02	b
100Affinity-2	136,471.8	135,996.2	0.60	0.35	10.14		136,171.7	0.22	10.02	b
25Chain-3	2855.2	2855.2	0.44	0.00	10.02	b,o	2855.2	0.00	0.70	b,o
25Graph-3	1935.7	1934.7	0.26	0.05	10.02	b	1919.4	0.84	0.62	
25Affinity-3	2002.3	2002.3	0.00	0.00	5.26	b,o	1994.9	0.37	0.84	
50Chain-3	6221.7	6094.3	4.36	2.05	10.02		6167.1	0.88	6.85	b
50Graph-3	5944.3	–	–	–	10.06		5918.4	0.44	4.52	b
50Affinity-3	5229.6	5189.1	2.15	0.77	10.03		5210.2	0.37	8.00	b
75Chain-3	12,206.2	–	–	–	10.11		12,048.9	1.29	10.23	b
75Graph-3	11,376.9	–	–	–	10.08		11,375.4	0.01	6.47	b
75Affinity-3	14,436.8	–	–	–	10.16		14,436.8	0.00	10.09	b
100Chain-3	19,337.9	–	–	–	10.09		19,336.0	0.01	10.39	b
100Graph-3	19,952.5	19,850.9	1.36	0.51	10.19		19,859.0	0.47	10.08	b
100Affinity-3	25,412.8	–	–	–	10.27		25,371.2	0.16	10.39	b
25Chain-4	18,240.4	18,240.4	0.00	0.00	1.25	b,o	18,240.4	0.00	1.84	b,o
25Graph-4	20,561.7	20,561.7	0.00	0.00	6.32	b,o	20,551.4	0.05	0.84	
25Affinity-4	20,308.8	20,308.8	0.00	0.00	1.81	b,o	20,286.8	0.11	0.45	
50Chain-4	60,490.4	60,153.6	2.85	0.56	10.03	b	60,033.9	0.75	9.39	
50Graph-4	66,394.0	66,099.6	1.36	0.44	10.05		66,166.2	0.34	7.46	b
50Affinity-4	60,073.0	59,604.8	1.23	0.78	10.05		59,912.3	0.27	10.02	b
75Chain-4	113,571.4	–	–	–	10.05		111,538.6	1.79	10.03	b
75Graph-4	130,590.7	128,261.4	3.26	1.78	10.08		129,963.0	0.48	10.14	b
75Affinity-4	124,913.5	123,522.5	2.68	1.11	10.05		123,763.8	0.92	10.02	b
100Chain-4	216,381.8	–	–	–	10.06		215,393.1	0.46	10.26	b
100Graph-4	246,939.9	241,622.6	3.42	2.15	10.16		246,505.4	0.18	10.28	b
100Affinity-4	237,691.5	232,747.4	3.43	2.08	10.12		236,441.2	0.53	10.16	b
Max/num			–	–	10.27	26,15		1.79	10.75	33,9
Average			–	–	8.06			0.34	6.94	
Average feasible			1.05	0.52	7.54			0.30	6.36	

generates only one more best solution than the version with $\gamma=1$, but it requires a larger computing time, because it executes procedure *GreedyHeuristic* a larger number of times. Since the average computing time with $\gamma=1$ is 36.70 s and with $\gamma=0.995$ is 59.65 s, whereas the average quality of solutions is practically the same, setting $\gamma=1$ is more convenient.

Column LGAP shows that the average gap between the lower and upper bounds (z^* and z_{LR}^*) provided by *LagrangianHeuristic* is 3.46% on average, which is not tight enough to give a reliable estimate of the quality of the heuristic solution.

Tables 4 and 5 compare *LagrangianHeuristic* with IBM Ilog Cplex setting the time limit to 60 and 10 s, respectively. In these computational tests we set $\gamma=1$.

Table 4 shows that, setting a time limit of 60 s, IBM Ilog Cplex generates the best solution for 31 out of 50 instances (21 of them are the best known and 19 are optimal) but it cannot find a feasible solution for 6 out of 50 instances. *LagrangianHeuristic* generates the best solution for 28 out of 50 instances (12 of them are the best known and 9 are optimal) and it finds a feasible solution for all instances. Moreover, the average gap between the value of the heuristic solution found and the best solution value known z_{best} for IBM Ilog Cplex is 0.48% and for *LagrangianHeuristic* is 0.26% (if we consider the instances where IBM Ilog Cplex had found a feasible solution).

Setting a time limit of 10 s, the situation worsens further for IBM Ilog Cplex. Table 5 shows that IBM Ilog Cplex generates the best solution for 26 instances (15 of them are the best known and

also optimal) but it cannot find a feasible solution for 10 out of 50 instances. *LagrangianHeuristic* generates the best solution for 33 instances (10 of them are the best known and 9 are optimal) and it still finds a feasible solution for all instances. Moreover, the average gap between the value of the heuristic solution found and the best solution value known z_{best} for IBM Ilog Cplex is 0.52% and for *LagrangianHeuristic* is 0.30%.

Overall, the results show that *LagrangianHeuristic* finds very good-quality solutions in a very short time with respect to IBM Ilog Cplex. Moreover, the procedure *LagrangianHeuristic* largely outperforms *GreedyHeuristic* even when we set a time limit of 10 s, since the average gap between the value of the heuristic solution found and z_{best} is four times smaller, i.e., 0.34% instead of 1.38% (remind that the gap of 1.38% is due to the 600 s time limit).

7. Concluding remarks

In this paper we have presented an approach for the sprint planning in agile methods based on an integer linear programming model. The mathematical formulation can be solved by a general purpose MIP solver like IBM Ilog Cplex. However, as shown in Section 6, the computing time to solve to optimality medium-size instances can be very large. In an attempt to reduce the computing time we have considered reduction procedures and some valid inequalities, integrated in IBM Ilog Cplex by means of the *callback functions*. The valid inequalities investigated are the classical *Lifted Cover Inequalities* and some *Dominance Inequalities*. This attempt has been done to show that the development of a heuristic approach is fully justified. In fact, even these enhancements are not enough to solve effectively the proposed model using a general purpose solver (see Section 6). However, reduction procedures and dominance inequalities are new contributions that could be useful for possible future developments and, also for this reason, we briefly describe them in this paper.

Since solving the model to optimality by a general-purpose MIP solver is not suitable for an operational use, we have proposed an effective Lagrangian heuristic. The computational results show that this Lagrangian heuristic is able to find good quality solutions very quickly (even within 10 s), while for some instances IBM Ilog Cplex also fails in finding a feasible solution even if the time limit is large (e.g., 600 s). The Lagrangian heuristic is effective and very stable: the average gap between the heuristic solution found and the best known is around the 0.3% and maximum gap is within the 1.8% even if the time limit is set equal to 10 s. Considering that the parameters used in the mathematical model are obtained by estimates, the quality of the solutions found by the Lagrangian heuristic is very satisfactory.

We close the paper by mentioning a couple of interesting directions for extending our approach. In practical project management, resources could be differentiated by skills. In particular, user story points could be defined for each skill, in which case the sprint capacity constraints would become

$$\sum_{j=1}^n p_{jk} r_j^{um} x_{ij} \leq p_{ik}^{max}, \quad i \in S, \quad k \in K \quad (39)$$

where K is the index set of the skills considered. This extension can be accommodated quite easily, because the subproblems become *multi-dimensional knapsack problems* that can be solved by a dynamic programming procedure.

Another issue occurs during the sprint review phase, when estimates may be updated and new stories may be added, which requires stories to be prioritized again. Sprint replanning should be made by applying a limited number of changes, so that the plan is not completely disrupted at each iteration. To deal with this desideratum we can add the following constraint to our

mathematical model:

$$\sum_{(i,j) \in X} x_{ij} \geq L \quad (40)$$

where X represents the subset of variables $\{x_{ij}\}$ set to one in the previous sprints planning and L is the minimum number of variables of subset X that do not have to change. The proposed procedure can be adapted by relaxing constraint (40) in a Lagrangian fashion; when subproblem SP_i (or SP'_i) is solved at each sprint i , in the greedy heuristics we can heuristically add the following constraint:

$$\sum_{(i,j) \in X} x_{ij} \geq L' \quad (41)$$

where either $L' = L/m$ (to ensure a uniform distribution of the story assignments that do not have to change) or L' is the difference between L and the story assignments already changed in the previous sprints (to ensure a higher replanning effectiveness).

References

- [1] Aalto University, SoberIT. Agilefant; 2011. (<http://www.agilefant.org>).
- [2] Abrahamsson P, Warsta J, Siponen MT, Ronkainen J. New directions on agile methods: a comparative analysis. In: Proceedings of the international conference on software engineering; 2003. p. 244–54.
- [3] Avella P, Boccia M, Vasilyev I. A computational study of exact knapsack separation for the generalized assignment problem. *Computational Optimization and Applications* 2010;45:543–55.
- [4] Beasley J. Lagrangian relaxation. In: Reeves C, editor. *Modern heuristic techniques for combinatorial problems*. Blackwell Scientific Publications; 1993. p. 243–303.
- [5] Beck K, et al. Manifesto for agile software development; 2001. (<http://agilemanifesto.org>).
- [6] Boschetti M, Hadjinconstantinou E, Mingozzi A. New upper bounds for the finite two-dimensional orthogonal non-guillotine cutting stock problem. *IMA Journal of Management Mathematics* 2002;13:95–119.
- [7] Boschetti M, Maniezzo V. Benders decomposition, Lagrangian relaxation and metaheuristic design. *Journal of Heuristics* 2009;15:283–312.
- [8] Boschetti M, Maniezzo V, Roffilli M. Decomposition techniques as metaheuristic frameworks. In: *Matheuristics, Hybridizing Metaheuristics and Mathematical Programming*. Springer, New York; 2009. p. 135–58.
- [9] Boschetti M, Maniezzo V, Roffilli M, Bolufe Rohler A. *Matheuristics: optimization, simulation and control*. In: Blesa M, Blum C, Di Gasparo L, Roli A, Samples M, Aouo S, editors. *Hybrid metaheuristics*, Lecture notes on computer science, vol. 5818. Springer; 2009. p. 171–7.
- [10] Boschetti M, Mingozzi A. The two-dimensional finite bin packing problem. Part I. New lower bounds for the oriented case. *4OR* 2003;1:27–42.
- [11] Boschetti MA, Montaletti L. An exact algorithm for the two-dimensional strip-packing problem. *Operations Research* 2010;58(November (6)):1774–91.
- [12] Cohn M. *User stories applied: for agile software development*. Addison-Wesley Professional; 2004.
- [13] Cohn M. *Agile estimating and planning*. Prentice-Hall; 2005.
- [14] Collabnet. ScrumWorks; 2011. (<http://www.danube.com>).
- [15] Dybå T, Dingsøyr T. Empirical studies of agile software development: a systematic review. *Information and Software Technology* 2008;50(9–10):833–59.
- [16] Golfarelli M, Rizzi S, Turricchia E. Sprint planning optimization in agile data warehouse design. In: *Proceedings of the DaWaK*. Vienna, Austria; 2012. p. 30–41.
- [17] Golfarelli M, Rizzi S, Turricchia E. Multi-sprint planning and smooth replanning: an optimization model. *Journal of Systems and Software* 2013;86(9):2357–70.
- [18] Greer D, Ruhe G. Software release planning: an evolutionary and iterative approach. *Information and Software Technology* 2004;46(4):243–53.
- [19] IBM. IBM ILOG CPLEX optimizer; 2011. (<http://www-01.ibm.com>).
- [20] Martello S, Toth P. *Knapsack problems: algorithm and computer implementation*. John Wiley and Sons Ltd.; 1990.
- [21] Nicols A. Agile planning, estimation and tracking; 2009. (<http://www.slide.net/andrewnichols/agile-planning-estimation-and-tracking>).
- [22] Schwaber K. SCRUM development process. In: *Proceedings of the OOPSLA*; 1995.
- [23] Scrum Methodology. Scrum effort estimation and story points; 2012. (<http://scrummethodology.com/scrum-effort-estimation-and-story-points>).
- [24] Shor N. *Minimization methods for non-differentiable functions*. Springer-Verlag; 1985.
- [25] Szoke A. Conceptual scheduling model and optimized release scheduling for agile environments. *Information and Software Technology* 2011;53(6):574–91.
- [26] ThoughtWorks Studios. Mingle: agile project management; 2011. (<http://www.thoughtworks-studios.com>).
- [27] van Valkenhoef G, Tervonen T, de Brock B, Postmus D. Quantitative release planning in extreme programming. *Information and Software Technology* 2011;53(11):1227–35.
- [28] VersionOne. 4th annual state of agile development survey results; 2009. (<http://www.versionone.com>).